10 CAPSTONE & MIDTERM PROJECT TOPICS: PYTHON FOR INFORMATION SECURITY

Target Audience: 2nd-Year Undergraduate Students in Information Security | **Duration:** 4 Weeks | **Format:** Groups of 5 Students

---

I. GENERAL REGULATIONS AND POLICIES

1. Academic Scope and AI Usage Policy

- **Controlled Lab Environment Only:** All projects must be conducted strictly within isolated academic lab environments, local virtual machines, officially approved public datasets, or systems explicitly owned or authorized for testing. Cloud accounts and infrastructure resources must be pre-approved by the instructor.
- **Strict Prohibition of Unauthorized Testing:** It is strictly prohibited to use any project code, tools, automated scripts, or infrastructure to scan, exploit, disrupt, attack, exfiltrate data from, or target external networks, university infrastructure, or unauthorized third parties **under any circumstances**.
- **Safe Testing Traffic:** When demonstrating attack-related concepts, use only controlled, simulated benign/malicious traffic within designated sandbox environments. Do not execute real malware binaries outside isolated sandboxes, and never target live production systems.
- **Academic AI Usage:** Generative AI tools (e.g., ChatGPT, Claude, Gemini, GitHub Copilot) may be utilized for documentation lookup, concept clarification, boilerplate code assistance, and debugging. However, students must thoroughly understand the algorithmic logic, architecture, data flows, APIs, parameters, security risks, and operational limits of all generated components.
- **Independent Verification:** Students must independently test, audit, and validate all AI-suggested code and experimental metrics. AI-generated reports or fabricated benchmark figures will not be accepted in place of real experimental results. During project defense, each team member must explain their specific contributions and live-code or troubleshoot a scenario assigned by the instructor.

2. Version Control and Team Collaboration Standards

- **Mandatory Git Repository:** All teams must use Git and host their source code on GitHub or GitLab. Teams must submit their repository URL and complete commit history. For private repositories, read access must be granted to the instructor.
- **Individual Contribution Tracking:** Every team member must commit using their own authenticated account with regular, well-documented commit messages. The instructor will verify individual contributions through commit logs, Pull Requests (PR) / Merge Requests (MR), and individual oral defense. Projects with only a single committer will incur severe individual score deductions.

- **Protected Trunk Workflow:** Direct commits or pushes to the main or master branch are strictly forbidden. All feature developments and bug fixes must be implemented on dedicated branches (feature/*, fix/*) and integrated via Pull Requests/Merge Requests accompanied by descriptive summaries and peer code reviews.
- **Branch Protection & Secrets Hygiene:** Configure branch protection rules where supported by the Git platform. Never commit API keys, cloud credentials, access tokens, private keys, or sensitive customer data into the repository. Use environment variables and .env.example templates.
- **Documentation Standards:** The repository README.md must include prerequisites, clean installation steps, runtime commands, automated test instructions, architecture overview, project directory structure, task breakdown per member, and authoritative reference links.

II. SUMMARY OF THE 10 CAPSTONE TOPICS

Topic Code	Project Title	Domain	Primary Technology Stack
01	Automated Deployment and Security Testing of AWS WAF using Pulumi (Python)	Cloud Security	Python 3, Pulumi Automation API, pulumi-aws, AWS WAFv2, CloudFront, S3, boto3 (logs)
02	Automated Deployment and Security Testing of Azure Front Door WAF using Pulumi (Python)	Cloud Security	Python 3, Pulumi Automation API, pulumi-azure-native, Azure Front Door, App Service, azure-monitor-query
03	Automated Deployment and Security Testing of Google Cloud Armor using Pulumi (Python)	Cloud Security	Python 3, Pulumi Automation API, pulumi-gcp, Cloud Armor, Cloud Run, google-cloud-logging
04	Windows PE Malware Detection using Machine Learning on BODMAS Dataset	AI / Malware Detection	Python 3, Pandas, Scikit-learn, LightGBM, FastAPI/Streamlit
05	Malware Family and Category Classification from Memory Artifacts (CIC-MalMem-2022)	AI / Malware Forensics	Python 3, Pandas, Scikit-learn, LightGBM, Streamlit/FastAPI

06	DevSecOps CI/CD Pipeline with Python Security Policy Engine	DevSecOps	Python 3, GitHub Actions, Bandit, pip-audit, Gitleaks, Trivy
07	Automated Security Assessment Toolkit with Web/Desktop GUI	Security Assessment / Pentest	Python 3, dnspython, Socket, SSL, OSV API / NVD, Streamlit / PySide6
08	Multi-Source SOC Analyst Toolkit with Web/Desktop GUI	Security Operations (SOC)	Python 3, Suricata EVE, Sysmon Parser, SQLite, FastAPI / Streamlit
09	NOC Infrastructure Monitoring Toolkit with Anomaly Detection	Network Operations (NOC)	Python 3, psutil, Prometheus Client, Scikit-learn, Streamlit / Dash
10	Kubernetes Security Monitoring and Automated Incident Response using Python	Cloud-Native Security	Python 3, Kubernetes Python Client, Kind/K3s, Calico CNI, SQLite

III. DETAILED PROJECT SPECIFICATIONS

COMMON CONSTRAINTS FOR TOPICS 01–03: Infrastructure as Code with Pulumi (Python)

Topics 01, 02, and 03 must provision **all** cloud infrastructure with **Pulumi**, written in Python and **driven from Python through the Pulumi Automation API** (pulumi.automation).

1. Allowed vs Forbidden Tooling

Allowed	Forbidden
Pulumi Python SDK (pulumi) and provider packages installed with pip: pulumi-aws, pulumi-azure-native, pulumi-gcp	Terraform/OpenTofu, AWS CDK, CDKTF, Bicep/ARM deployments, Deployment Manager
Pulumi CLI binary **as the engine only**, invoked by the Automation API (it may be installed from Python with auto.PulumiCommand.install())	Typing pulumi up / pulumi destroy manually as deployment evidence
Cloud CLIs (aws, az, gcloud) **only for login/authentication**	Creating, changing, or deleting resources with cloud CLIs or in the web console
Cloud SDKs (boto3, azure-monitor-query, google-cloud-logging) for **reading logs**	Shell scripts wrapping any forbidden tool

and verification	
pytest with Pulumi mocks (pulumi.runtime.set_mocks), pyyaml, pydantic, httpx, rich	Committing passphrases, keys, or state files to Git

The web console may be used **read-only** for screenshots. Any manually created or edited resource invalidates the deployment evidence.

2. Mandatory Architecture

- **Inline Pulumi program** (infra/program.py): a Python function that declares all resources and exports outputs with pulumi.export(...).
- **Declarative specification** (spec.yaml): WAF rules, modes, thresholds, blocked paths/IPs, region, SKU. It is validated with pydantic and passed to the program, so rule changes never require editing Python code.
- **Automation API driver** (python -m infra <command>):
- preview: stack.preview(), prints planned creates/updates/deletes.
- up: stack.up(), prints outputs and the resource-change summary.
- refresh: stack.refresh(), detects drift between Pulumi state and real cloud resources.
- status: prints stack outputs (stack.outputs()) and resource count.
- destroy: stack.destroy(), then stack.workspace.remove_stack(...) and SDK-based verification that nothing remains.
- **State backend:** local file backend (auto.ProjectBackend(url="file://./.pulumi-state")), with .pulumi-state/ git-ignored. Secrets are encrypted with a passphrase supplied through the environment variable PULUMI_CONFIG_PASSPHRASE (never hard-coded). Pulumi Cloud (free individual tier) is an allowed alternative.
- **Idempotency proof:** running up twice in a row must report only unchanged (same) resources the second time.
- **Tagging:** every taggable resource carries project=ptit-pyis and team=<id> (labels on GCP).
- **Budget guard:** status warns when the stack has been alive longer than 90 minutes (timestamp stored at up).
- **Offline unit tests:** pytest tests run the program with Pulumi mocks to assert, without any cloud account, that the expected rules, priorities, scopes, and tags are declared.

3. Minimal Automation API Skeleton (reference)

```
from pulumi import automation as auto

def get_stack(program, project="ptit-waf", stack="dev"):
    settings = auto.ProjectSettings(
        name=project, runtime="python",
        backend=auto.ProjectBackend(url="file://./.pulumi-state"),
```

```
    )
    return auto.create_or_select_stack(
        stack_name=stack, project_name=project, program=program,
        opts=auto.LocalWorkspaceOptions(project_settings=settings),
    )

# stack = get_stack(program)
# stack.set_config("aws:region", auto.ConfigValue("us-east-1"))
# stack.preview(on_output=print); result = stack.up(on_output=print)
# print(result.outputs, result.summary.resource_changes)
# stack.destroy(on_output=print)
```

4. Common Repository Layout

```
infra/
  __main__.py       # Automation API driver: preview | up | refresh | status | destroy
  program.py        # inline Pulumi program (resource declarations)
  spec.py           # pydantic models for spec.yaml
spec.yaml
security_tests/     # HTTP benign/attack test suite (httpx)
analysis/           # log retrieval with cloud SDK + metrics
tests/              # pytest unit tests with Pulumi mocks
reports/
```

TOPIC 01: Automated Deployment and Security Testing of AWS WAF using Pulumi (Python)

Context & Problem Statement

Cloud web applications face constant SQL injection (SQLi), Cross-Site Scripting (XSS), scanner, and Layer 7 flood traffic. Configuring WAFs by hand in the console leads to errors and drift. Teams will use **Pulumi with pulumi-aws**, driven by the Automation API, to declare a private S3 origin, a CloudFront distribution, and an AWS WAFv2 Web ACL with logging, then run an automated security test suite and analyze WAF logs.

A. Core Concepts to Master

- **Pulumi resource model:** resources, inputs/outputs (pulumi.Output, .apply()), dependency graph, stack state, preview/up/refresh/destroy.
- **Explicit providers:** a Web ACL for CloudFront must use scope="CLOUDFRONT" and be created in **us-east-1**; declare aws.Provider("use1", region="us-east-1") and pass it with pulumi.ResourceOptions(provider=...).
- **Key pulumi-aws resources:** aws.s3 bucket + bucket objects + bucket policy, aws.cloudfront.OriginAccessControl, aws.cloudfront.Distribution, aws.wafv2.IpSet, aws.wafv2.WebAcl, aws.cloudwatch.LogGroup, aws.wafv2.WebAclLoggingConfiguration.
- **Origin Access Control (OAC):** the bucket remains private; only CloudFront can read it.
- **Rule actions:** managed rule groups, rate-based rules, custom byte/regex match rules; actions allow, block, count, captcha.
- **WAF logging constraint:** the CloudWatch Logs group name **must start with aws-waf-logs-**.

B. Functional Requirements

- **Pulumi-declared infrastructure:**

1. Private S3 bucket with force_destroy=True, test content uploaded as Pulumi bucket objects (index.html, a few static assets), OAC, and a bucket policy that allows only the distribution.
2. CloudFront distribution with the S3 origin and web_acl_id set to the Web ACL ARN.
3. WAFv2 Web ACL (scope="CLOUDFRONT", us-east-1 provider) with at least 5 rules generated from spec.yaml:

- AWSManagedRulesSQLiRuleSet.
- AWSManagedRulesCommonRuleSet (includes XSS protections).
- Custom path rule blocking /admin, /.git, /.env.
- IP set block list (CIDR) or geo-match rule.
- Rate-based rule (for example 100 requests per 5 minutes per IP).

4. Log group aws-waf-logs-<team> + WAF logging configuration.
5. Exports: distribution domain, Web ACL ARN, bucket name, log group name.

- **Rule lifecycle:** switching any rule between count and block only by editing spec.yaml and running preview + up.
- **Automated security test suite** (httpx), at least 20 tagged cases: 8 benign, 4 SQLi, 4 XSS, 2 restricted paths, 2 rate-limit bursts. Each request carries a unique test ID for log correlation.
- **Telemetry analysis:** read the WAF log group with boto3 (filter_log_events or Logs Insights start_query/get_query_results), join with test IDs, compute True Positive Rate, False Positive Rate, and latency p50/p95 for Count vs Block runs.
- **Drift exercise:** the instructor (or a teammate) changes one WAF rule in the console; refresh must detect it and up must restore the declared state.

C. Step-by-Step Implementation Guide

Step 1: Environment

- python -m venv .venv then pip install pulumi pulumi-aws boto3 pyyaml pydantic httpx pytest rich
- Ensure the Pulumi CLI is available (or install it from Python with auto.PulumiCommand.install()).
- AWS credentials through a named profile (~/.aws/credentials) or AWS Academy Learner Lab keys; set stack config aws:profile and aws:region from the driver. Check identity with boto3 sts.get_caller_identity().

Step 2: Spec and validation

- Write spec.yaml (rule names, priorities, modes, rate limit, CIDRs, paths) and its pydantic

model; reject duplicate names/priorities and invalid rate limits.

Step 3: Write the inline Pulumi program

- infra/program.py reads the validated spec and declares the us-east-1 provider, bucket, objects, OAC, bucket policy, IP set, Web ACL, log group, logging configuration, and distribution, then exports outputs.
- Build the bucket policy JSON with pulumi.Output.all(...).apply(...) so it references the distribution ARN.

Step 4: Unit tests with mocks

- tests/test_program.py uses pulumi.runtime.set_mocks to assert that the Web ACL scope is CLOUDFRONT, that 5 or more rules exist with the expected actions, and that the log group name starts with aws-waf-logs-.

Step 5: Preview and deploy

- python -m infra preview → review planned changes.
- python -m infra up → CloudFront usually needs 3 to 10 minutes. Save the outputs in the report.
- Run up again and show that all resources are same.

Step 6: Run the security tests

- python -m security_tests --target https://<distribution-domain>
- Expected: benign → 200; SQLi/XSS/restricted paths → 403; rate burst → 403 after the threshold (rate-based rules react with a delay of tens of seconds, so the suite must send sustained traffic and record when blocking began).

Step 7: Count vs Block analysis and drift

- Run the suite with all rules in count, switch to block via spec.yaml + up, run again, then python -m analysis.
- Perform the drift exercise with refresh + up.

Step 8: Destroy and verify

- python -m infra destroy → remove the stack → confirm with boto3 that the Web ACL, distribution, and bucket no longer exist.

D. Deliverables

- Repository following the common layout (infra/program.py, driver, spec.yaml, tests).
- Preview output for at least one rule change, idempotency output (second up), drift-detection output.
- Report (Markdown + CSV): confusion matrix, TPR, FPR, latency p50/p95 for Count vs Block.
- Screen recording: preview → up → tests → rule change → destroy → verification.

E. Evaluation & Oral Defense

- Live change requested by the instructor (new blocked path, new rate limit) via spec.yaml, with preview diff and proven enforcement.
- Explain Pulumi Outputs and .apply(), why an explicit us-east-1 provider is needed, and how OAC protects the origin.

F. Official References

- Pulumi Automation API: https://www.pulumi.com/docs/iac/using-pulumi/automation-api/
- Pulumi AWS provider: https://www.pulumi.com/registry/packages/aws/
- aws.wafv2.WebAcl: https://www.pulumi.com/registry/packages/aws/api-docs/wafv2/webacl/
- Pulumi unit testing: https://www.pulumi.com/docs/iac/concepts/testing/unit/
- AWS WAF logging to CloudWatch Logs: https://docs.aws.amazon.com/waf/latest/developerguide/logging-cw-logs.html
- AWS WAF pricing: https://aws.amazon.com/waf/pricing/

TOPIC 02: Automated Deployment and Security Testing of Azure Front Door WAF using Pulumi (Python)

Context & Problem Statement

Azure Front Door (AFD) is Microsoft's global edge load balancer and CDN; its WAF policies inspect HTTP traffic before it reaches the backend. Teams will use **Pulumi with pulumi-azure-native**, driven by the Automation API, to declare an App Service backend, an AFD Standard/Premium profile, a WAF policy, origin cloaking, and diagnostic logging, then compare Detection vs Prevention modes.

A. Core Concepts to Master

- **pulumi-azure-native:** resources map directly to Azure Resource Manager APIs.
- **Key resources:**
- resources.ResourceGroup.
- web.AppServicePlan (Linux) and web.WebApp (container image, access restrictions).
- cdn.Profile, cdn.AFDEndpoint, cdn.AFDOriginGroup, cdn.AFDOrigin, cdn.Route, cdn.SecurityPolicy.
- frontdoor.Policy (Front Door WAF policy).
- operationalinsights.Workspace and insights.DiagnosticSetting.
- **SKU rules:** the WAF policy SKU must match the profile SKU (Standard_AzureFrontDoor or Premium_AzureFrontDoor). Managed rule sets (Microsoft Default Rule Set 2.1, Bot Manager) require **Premium**; Standard supports custom rules only.

- **WAF modes:** Detection (log only) and Prevention (block with HTTP 403).
- **Origin cloaking:** Web App access restrictions allow only the AzureFrontDoor.Backend service tag **and** require header X-Azure-FDID equal to the profile's front_door_id output.

B. Functional Requirements

- **Pulumi-declared infrastructure:**

1. Resource group.
2. Linux App Service plan (B1) + Web App running a public sample image (for example mcr.microsoft.com/azuredocs/aci-helloworld).
3. AFD profile, endpoint, origin group (with health probe), origin (Web App hostname), and route.
4. WAF policy (frontdoor.Policy) generated from spec.yaml:

- Premium: Microsoft Default Rule Set 2.1 + custom rules.
- Standard (budget option): custom rules for SQLi/XSS patterns (match conditions with transforms such as UrlDecode, Lowercase), path blocking, IP/geo blocking, and a rate-limit rule (for example 100 requests per 5 minutes per client IP).

5. Security policy binding the WAF policy to the endpoint.
6. Web App access restriction using the AzureFrontDoor.Backend service tag and header x-azure-fdid = profile.front_door_id.
7. Log Analytics workspace + diagnostic setting on the profile for FrontDoorAccessLog and FrontDoorWebApplicationFirewallLog.
8. Exports: endpoint hostname, Web App hostname, workspace ID, Front Door ID.

- **Mode lifecycle:** Detection ↔ Prevention only via spec.yaml + preview + up.
- **Automated security test suite** (httpx), at least 20 tagged cases: 8 benign, 4 SQLi, 4 XSS, 2 path/origin-bypass, 2 rate-limit bursts. Direct calls to *.azurewebsites.net must receive HTTP 403.
- **Telemetry analysis:** query the workspace with azure-monitor-query (LogsQueryClient.query_workspace), for example AzureDiagnostics | where Category == "FrontDoorWebApplicationFirewallLog", and compute TPR, FPR, and latency for both modes. Allow for ingestion delay (often 5 to 15 minutes).
- **Drift exercise:** a manual change to the WAF mode in the console must be detected by refresh and reverted by up.

C. Step-by-Step Implementation Guide

Step 1: Environment and authentication

- python -m venv .venv then pip install pulumi pulumi-azure-native azure-identity azure-monitor-query pyyaml pydantic httpx pytest rich
- Authenticate with az login (login only) or a service principal through the environment variables ARM_CLIENT_ID, ARM_CLIENT_SECRET, ARM_TENANT_ID,

ARM_SUBSCRIPTION_ID. Set azure-native:location from the driver.

Step 2: Spec and validation

- spec.yaml: location, SKU, WAF mode, custom rules, rate limit, blocked paths/IPs, container image; reject managed rule sets when SKU is Standard.

Step 3: Write the inline Pulumi program

- Declare resources in infra/program.py. For the Web App, set site_config with linux_fx_version="DOCKER|<image>" and ip_security_restrictions whose headers value uses profile.front_door_id (a Pulumi Output, so the dependency is automatic).

Step 4: Unit tests with mocks

- Assert that the WAF policy SKU equals the profile SKU, that the policy mode comes from the spec, and that the access restriction includes the x-azure-fdid header.

Step 5: Preview and deploy

- python -m infra preview then python -m infra up. AFD propagation can take several minutes; the driver polls the endpoint with httpx until it returns 200.
- Run up again to prove idempotency.

Step 6: Security tests and origin-cloaking proof

- python -m security_tests --target https://<endpoint>.azurefd.net --origin https://<app>.azurewebsites.net

Step 7: Detection vs Prevention, drift

- Run in Detection, switch to Prevention via spec.yaml + up, run again, wait for ingestion, then python -m analysis. Perform the drift exercise.

Step 8: Destroy and verify

- python -m infra destroy → remove the stack → confirm with azure-identity + a simple ARM check (or the Pulumi stack resource count) that no resources remain.

D. Deliverables

- Repository following the common layout.
- Preview output before/after a mode change, idempotency output, drift-detection output.
- Report: origin-cloaking evidence, confusion matrix, TPR/FPR, latency for Detection vs Prevention.
- Screen recording of the full lifecycle ending with verified deletion.

E. Evaluation & Oral Defense

- Live rule or mode change through spec.yaml, preview diff, observed enforcement.
- Explain X-Azure-FDID, why WAF and profile SKUs must match, and how Pulumi derives the dependency between the profile and the Web App restriction.

F. Official References

- Pulumi Azure Native provider: https://www.pulumi.com/registry/packages/azure-native/
- azure-native.cdn.Profile: https://www.pulumi.com/registry/packages/azure-native/api-docs/cdn/profile/
- azure-native.frontdoor.Policy: https://www.pulumi.com/registry/packages/azure-native/api-docs/frontdoor/policy/
- Azure Front Door WAF overview: https://learn.microsoft.com/en-us/azure/web-application-firewall/afds/afds-overview
- Secure origin with Front Door: https://learn.microsoft.com/en-us/azure/frontdoor/origin-security
- Azure Front Door pricing: https://azure.microsoft.com/en-us/pricing/details/frontdoor/

TOPIC 03: Automated Deployment and Security Testing of Google Cloud Armor using Pulumi (Python)

Context & Problem Statement

Google Cloud Armor provides WAF and DDoS protection for services behind the Global External Application Load Balancer. Teams will use **Pulumi with pulumi-gcp**, driven by the Automation API, to declare a Cloud Run backend, a serverless NEG, a global HTTP load balancer, and a Cloud Armor security policy, then compare Preview vs Enforce behavior.

A. Core Concepts to Master

- **Key pulumi-gcp resources:**
- projects.Service (enable compute, run, logging APIs).
- cloudrunv2.Service + cloudrunv2.ServiceIamMember (public invoker).
- compute.RegionNetworkEndpointGroup (network_endpoint_type="SERVERLESS").
- compute.SecurityPolicy (rules with priority, match, action, preview).
- compute.BackendService (load_balancing_scheme="EXTERNAL_MANAGED", security_policy, log_config).
- compute.URLMap, compute.TargetHttpProxy, compute.GlobalAddress, compute.GlobalForwardingRule.
- **Preconfigured WAF expressions:** evaluatePreconfiguredExpr('sqli-v33-stable'), evaluatePreconfiguredExpr('xss-v33-stable').
- **Preview mode:** per-rule preview=True logs the decision without enforcing it.
- **Rate limiting:** throttle or rate_based_ban actions with deny(429) as the exceed action.
- **Ingress restriction:** Cloud Run ingress INGRESS_TRAFFIC_INTERNAL_LOAD_BALANCER rejects direct *.run.app requests.

- **Logging:** Cloud Armor decisions (jsonPayload.enforcedSecurityPolicy, previewSecurityPolicy) appear in load balancer logs only when backend-service logging is enabled.

B. Functional Requirements

- **Pulumi-declared infrastructure:**

1. Required APIs enabled through projects.Service.
2. Cloud Run service from us-docker.pkg.dev/cloudrun/container/hello, ingress restricted to internal + load balancer, public invoker binding.
3. Serverless NEG for the Cloud Run service.
4. Cloud Armor security policy generated from spec.yaml:

- SQLi and XSS preconfigured rules with deny(403).
- Custom path rule (for example request.path.matches('/admin|/\\.git|/\\.env')).
- IP block rule.
- Rate-limit rule (throttle, for example 100 requests per 60 seconds per IP, exceed action deny(429)).
- Default rule (priority 2147483647) set to allow.

5. Backend service with the NEG, logging enabled (sample rate 1.0), and the security policy attached.
6. URL map, target HTTP proxy, global IP address, forwarding rule on port 80.
7. Exports: load balancer IP, Cloud Run URL, policy name.

- **Preview lifecycle:** toggling preview only via spec.yaml + preview + up.
- **Automated security test suite** (httpx), at least 20 tagged cases: 8 benign, 4 SQLi, 4 XSS, 2 path/IP, 2 rate-limit bursts; plus an ingress test calling the *.run.app URL directly, which must be rejected (record the exact HTTP status observed).
- **Telemetry analysis:** google-cloud-logging list_entries with a filter such as resource.type="http_load_balancer" AND jsonPayload.enforcedSecurityPolicy.name="<policy>", joined with test IDs; compute TPR, FPR, and latency for Preview vs Enforce.
- **Drift exercise:** a manual console change to a rule priority must be detected by refresh and reverted by up.

C. Step-by-Step Implementation Guide

Step 1: Environment and authentication

- python -m venv .venv then pip install pulumi pulumi-gcp google-cloud-logging pyyaml pydantic httpx pytest rich
- Authenticate with gcloud auth application-default login (login only) or a service-account key file referenced by GOOGLE_APPLICATION_CREDENTIALS and stored outside the

repository. Set gcp:project and gcp:region from the driver.

Step 2: Spec and validation

- spec.yaml: project, region, image, rules (priority, expression, action, preview), rate-limit settings, blocked IPs/paths; reject duplicate priorities and use of the reserved default priority.

Step 3: Write the inline Pulumi program

- Declare resources in infra/program.py. Use depends_on so Cloud Run and Compute resources wait for the projects.Service API enablement.

Step 4: Unit tests with mocks

- Assert that SQLi/XSS rules use the preconfigured expressions, that the default rule allows, that backend logging is enabled, and that Cloud Run ingress is restricted.

Step 5: Preview and deploy

- python -m infra preview then python -m infra up. The load balancer can take several minutes to serve traffic; the driver polls the IP until HTTP 200.
- Run up again to prove idempotency.

Step 6: Security tests and ingress proof

- python -m security_tests --target http://<lb-ip> --origin https://<service>.run.app

Step 7: Preview vs Enforce, drift

- Run with all WAF rules in preview, switch to enforce via spec.yaml + up, run again, then python -m analysis. Perform the drift exercise.

Step 8: Destroy and verify

- python -m infra destroy → remove the stack → verify through google-cloud-compute or the console (read-only) that the forwarding rule, backend service, policy, and Cloud Run service are gone.

D. Deliverables

- Repository following the common layout.
- Preview output before/after a preview toggle, idempotency output, drift-detection output.
- Report: ingress-restriction evidence, confusion matrix, TPR/FPR, latency for Preview vs Enforce.
- Screen recording of the full lifecycle and post-destroy verification.

E. Evaluation & Oral Defense

- Live change: change a rule priority or toggle preview via spec.yaml, show the diff, apply, and show the new decision in logs.
- Explain the serverless NEG, why backend logging must be enabled, and how Pulumi orders resource creation and deletion.

F. Official References

- Pulumi Google Cloud provider: https://www.pulumi.com/registry/packages/gcp/
- gcp.compute.SecurityPolicy: https://www.pulumi.com/registry/packages/gcp/api-docs/compute/securitypolicy/
- Cloud Armor security policy overview: https://cloud.google.com/armor/docs/security-policy-overview
- Cloud Armor preconfigured WAF rules: https://cloud.google.com/armor/docs/waf-rules
- Cloud Armor rate limiting: https://cloud.google.com/armor/docs/rate-limiting-overview
- Serverless NEG with Cloud Run: https://cloud.google.com/load-balancing/docs/https/setup-global-ext-https-serverless

---

TOPIC 04: Windows PE Malware Detection using Machine Learning on BODMAS Dataset

Context & Problem Statement

Antivirus and Endpoint Detection & Response (EDR) agents increasingly employ machine learning models to detect malicious Windows Portable Executable (PE) binaries before execution. The BODMAS dataset (published by security researchers from Georgia Tech and McAfee) provides a rigorously curated, temporally tagged collection of PE static features. Students will construct a machine learning pipeline in Python to preprocess feature vectors, train classification models, mitigate data leakage, tune decision thresholds, and deploy an inference API.

A. Core Concepts to Master

- **Windows Portable Executable (PE):** Standard file executable format (.exe, .dll, .sys) in Windows operating systems.
- **Static Feature Vectors:** Numeric representations of PE structure (headers, section characteristics, entropy, import tables, byte histograms) extracted without executing the binary (using LIEF/EMBER extractors, yielding 2,381 features).
- **Data Leakage & Temporal Splitting:** In security ML, randomly splitting data across time introduces severe data leakage because future malware variants leak into the training set. Proper validation relies on temporal splits (training on older samples, testing on newer samples).
- **Precision-Recall AUC (PR-AUC):** Essential evaluation metric for imbalanced security datasets where false alarms severely impact SOC operations.
- **False Positive Rate (FPR) Thresholding:** In enterprise AV deployment, a model with 99% accuracy but 1% FPR is unusable because it blocks thousands of legitimate business applications. Thresholds must be chosen to guarantee FPR below 0.1% or 1%.

B. Functional Requirements

- **Dataset Ingestion:** Download the official BODMAS dataset feature vectors (.npy or .csv) and metadata. Select a representative subset of **20,000 to 40,000 samples** to ensure training runs efficiently on standard student laptops (< 16 GB RAM).
- **Data Preprocessing & Audit:** Inspect feature distributions, eliminate constant/zero-variance columns, handle missing values, and scale features using StandardScaler or RobustScaler (fit strictly on the training set).
- **Model Training:** Train, benchmark, and compare at least three distinct algorithms:

1. Baseline: Logistic Regression or Linear SVM.
2. Ensemble: Random Forest.
3. Gradient Boosting: LightGBM (recommended for sparse tabular data) or XGBoost.

- **Rigorous Evaluation:**

1. Evaluate models on an isolated test set using: Precision, Recall, F1-Score, ROC-AUC, PR-AUC, and FPR.
2. Perform threshold analysis: Determine the optimal probability threshold that maintains FPR below 0.5%.
3. Perform error analysis on at least 10 false positives (clean files flagged as malware) and 10 false negatives (malware missed).

- **Inference Service:** Package the best-performing model into an interactive Web UI (Streamlit) or REST API (FastAPI) allowing users to upload a feature CSV/JSON and receive real-time classification, risk probability, and model version metadata.

C. Step-by-Step Implementation & Feasibility Notes

- **Safety Guarantee:** Students **do not download or execute live malware binaries**. The project uses solely pre-extracted numerical feature matrices (.npz / .csv) and metadata.
- **Hardware Feasibility:** Training on 30,000 samples with LightGBM takes less than 2 minutes on a standard 4-core CPU. Use n_jobs=-1 and set max_depth=8, n_estimators=100 to prevent memory exhaustion.

D. Deliverables

- Clean Python data pipeline: download.py, preprocess.py, train.py, evaluate.py, app.py.
- Saved model artifacts (.joblib or .onnx) accompanied by feature schema documentation.
- Evaluation dashboard (Streamlit) displaying confusion matrices, ROC/PR curves, and interactive prediction.
- Technical report detailing data split methodology, hyperparameter choices, and threshold trade-offs.

E. Evaluation & Oral Defense Criteria

- Live inference: The instructor will feed 5 unseen feature vectors to the API and evaluate the predicted class and confidence.

- Defend the choice of probability threshold and explain why accuracy is an inadequate metric in malware detection.
- Demonstrate that feature scaling parameters were not leaked from the test set.

F. Official References

- BODMAS Research Paper & Repository: https://github.com/whyisyoung/BODMAS
- EMBER 2018 PE Feature Extractor: https://github.com/elastic/ember
- Scikit-learn Classifier Threshold Tuning: https://scikit-learn.org/stable/modules/classification_threshold.html

---

TOPIC 05: Malware Family and Category Classification from Memory Artifacts (CIC-MalMem-2022)

Context & Problem Statement

Fileless malware, memory-resident payloads, and code-injection attacks evade conventional disk-based antivirus scanners. Memory forensics captures volatile system states to expose malicious processes, DLL injections, and hooking mechanisms. The CIC-MalMem-2022 dataset from the Canadian Institute for Cybersecurity contains 58,596 memory dump feature records extracted via Volatility. In this project, students build an end-to-end Python machine learning classifier to distinguish benign memory states from distinct malware categories (Ransomware, Spyware, Trojan Horse) and implement prediction rejection under uncertainty.

A. Core Concepts to Master

- **Memory Forensics Artifacts:** Volatile indicators including process handles, DLLs loaded from unmapped memory, process tree anomalies, API hooks, and memory section permissions (e.g., PAGE_EXECUTE_READWRITE).
- **Multiclass Classification:** Categorizing memory dumps into Benign, Ransomware, Spyware, and Trojan Horse.
- **Macro-F1 Score:** Calculating unweighted average F1 across all classes to ensure minority malware classes are evaluated fairly.
- **Confidence Calibration & Prediction Rejection (Uncertainty Handling):** In clinical and security triage, when model prediction confidence falls below an established confidence margin (e.g., maximum softmax probability < 0.65), the system outputs Uncertain / Human Triage Required rather than making an unreliable guess.

B. Functional Requirements

- **Dataset Audit:** Load the official CIC-MalMem-2022 dataset (58,596 rows, 56 numerical features). Audit column distributions, drop non-predictive identifiers (such as process names or image filenames that cause trivial shortcut learning), and verify class balance.
- **Preprocessing Pipeline:** Construct a Scikit-learn pipeline implementing missing-value

handling, outlier detection, and feature scaling (StandardScaler or MinMaxScaler).

- **Model Development:**
- Task 1: Binary Classification (Benign vs. Malicious).
- Task 2: Multiclass Classification (Benign vs. Ransomware vs. Spyware vs. Trojan).
- Train at least two distinct architectures: Random Forest and LightGBM / Gradient Boosting.
- **Uncertainty & Triage Module:**
- Implement a threshold mechanism: If the highest predicted probability is below threshold tau, classify as Uncertain.
- Benchmark Coverage vs. Accuracy: Demonstrate how discarding uncertain predictions improves accuracy on remaining samples.
- **Web Application / Dashboard:** Build a Streamlit or FastAPI application allowing security analysts to upload memory feature vectors, view predicted category, risk scores across all classes, and top-3 contributing features.

C. Step-by-Step Implementation & Feasibility Notes

- **Hardware Feasibility:** The full 58,596-row tabular dataset is approximately 12 MB in CSV format. Training takes under 60 seconds on any modern CPU with 8 GB RAM.
- **Safety:** No raw memory dumps (.raw / .vmem) or malware binaries are handled; all analysis is conducted on structured tabular feature matrices.

D. Deliverables

- Complete Python repository: audit.py, train.py, evaluate.py, app.py.
- Trained model files (.pkl / .joblib) and test split datasets.
- Interactive Web GUI showing category predictions, probability bar charts, and uncertainty flags.
- Comprehensive technical report containing confusion matrices, per-class Precision/Recall/F1, and feature importance rankings.

E. Evaluation & Oral Defense Criteria

- Live test: The instructor will input memory feature samples; the team must explain the predicted category and confidence score.
- Demonstrate the prediction rejection feature by showing a sample that triggers the Uncertain label.
- Explain which memory features (e.g., malfind, handles, ldrmodules) contributed most heavily to detecting Trojan activity.

F. Official References

- CIC-MalMem-2022 Official UNB Page: https://www.unb.ca/cic/datasets/malmem-2022.html

- Volatility 3 Memory Forensics Framework: https://volatility3.readthedocs.io/en/latest/
- Scikit-learn Multiclass Metrics: https://scikit-learn.org/stable/modules/model_evaluation.html#multiclass-and-multilabel-classification#multiclass-and-multilabel-classification

---

TOPIC 06: DevSecOps CI/CD Pipeline with Python Security Policy Engine

Context & Problem Statement

Modern software engineering shifts security left by integrating automated scanning into Continuous Integration / Continuous Deployment (CI/CD) pipelines. However, running disjointed security scanners produces overwhelming noise, disparate report formats, duplicate alerts, and unmanageable false positives. In this project, students develop a complete DevSecOps pipeline on GitHub Actions or GitLab CI, centered around a custom **Python Security Policy Engine** that parses heterogeneous scanner outputs, unifies them into a standard schema, deduplicates findings, and enforces custom pass/fail security gates.

A. Core Concepts to Master

- **Static Application Security Testing (SAST):** Analyzing source code for security flaws (e.g., SQLi, command injection, hardcoded credentials) without running the program. Tool: Bandit.
- **Software Composition Analysis (SCA):** Scanning third-party dependencies against vulnerability databases (CVEs). Tool: pip-audit.
- **Secret Scanning:** Detecting committed API keys, tokens, and private keys using regex and entropy analysis. Tool: Gitleaks.
- **Container / IaC Scanning:** Detecting OS package vulnerabilities and Dockerfile misconfigurations. Tool: Trivy.
- **SARIF (Static Analysis Results Interchange Format):** Standard JSON format for static analysis scanner outputs.
- **Declarative Policy Gate (policy.yaml):** Business logic dictating build success or failure based on severity thresholds, vulnerability age, and approved temporary exceptions.

B. Functional Requirements

- **Sample Vulnerable Web Application:** Create a sample Python Flask or FastAPI web service with two deliberate test states:
- Branch vulnerable: Contains deliberate flaws (e.g., hardcoded test secret, outdated vulnerable package requests==2.20.0, insecure eval() call).
- Branch secure: Remediated version fixing all reported issues.
- **Automated CI/CD Workflow:** Implement GitHub Actions (.github/workflows/security.yml) or GitLab CI running:

- SAST with Bandit (exporting JSON/SARIF).
- SCA with pip-audit (exporting JSON).
- Secret scanning with Gitleaks (exporting JSON/SARIF).
- Container vulnerability scanning with Trivy (exporting JSON/SARIF).
- **Custom Python Policy Engine:** Write a standalone Python program that:
- Parses raw JSON/SARIF outputs from all 4 scanners.
- Normalizes findings into a unified data structure (tool, file, line, rule_id, severity, title).
- Deduplicates identical findings across scanners.
- Reads a declarative policy.yaml (e.g., fail_on: CRITICAL, HIGH, max_allowed_medium: 2, exceptions: [CVE-XXXX-YYYY with expiration date]).
- Computes policy compliance: Returns exit code 0 (Pass) if within policy, or exit code 1 (Fail) to abort pipeline deployment.
- **Reporting:** Export an aggregated HTML and Markdown summary directly to GitHub Actions Job Summary.

C. Step-by-Step Implementation & Feasibility Notes

- **Zero Cloud Cost:** Runs 100% free within GitHub Actions free runner minutes (2,000 minutes/month).
- **Tooling Verification:** bandit, pip-audit, gitleaks, and trivy are industry-standard, fully open-source tools with native Linux/CI support.

D. Deliverables

- Repository with sample web app, .github/workflows/security.yml, and security_engine/ Python package.
- Declarative policy.yaml supporting temporary vulnerability waivers with expiration dates.
- CI/CD execution proof:
- Run 1 on vulnerable branch: Pipeline successfully fails and generates detailed vulnerability breakdown.
- Run 2 on secure branch: Pipeline succeeds and allows deployment artifact generation.
- Aggregated HTML security report.

E. Evaluation & Oral Defense Criteria

- Live test: The instructor will inject a mock CVE or a secret string into a PR; students must show the pipeline blocking the PR.
- Add an exception with an expiration date in policy.yaml and demonstrate how the policy engine handles valid vs. expired waivers.
- Explain the parser normalization logic for SARIF/JSON schemas.

F. Official References

- Bandit Documentation: https://bandit.readthedocs.io/en/latest/
- pip-audit by PyPA: https://pypi.org/project/pip-audit/
- Gitleaks Official Repository: https://github.com/gitleaks/gitleaks
- Trivy Security Scanner: https://trivy.dev/latest/
- OASIS SARIF Standard: https://sarifweb.azurewebsites.net/

TOPIC 07: Automated Security Assessment Toolkit with Web/Desktop GUI

Context & Problem Statement

Security analysts and penetration testers regularly perform external asset discovery, port enumeration, HTTP security posture verification, and vulnerability enrichment. Rather than juggling dozens of disparate command-line scripts, security teams require consolidated, modular toolkits with clear scope controls to avoid illegal scanning. In this project, students develop an automated Python Security Assessment Toolkit featuring a Web (Streamlit/FastAPI) or Desktop (PySide6) graphical interface, strict allowlist enforcement, modular scan engines, and structured reporting.

A. Core Concepts to Master

- **Scope & Allowlist Enforcement:** Strict guardrail mechanism preventing scans against unauthorized domains or public IP blocks.
- **DNS Enumeration:** Discovering IP addresses, mail exchangers, and name servers using dnspython (A, AAAA, MX, NS, TXT records).
- **Service & Port Discovery:** Fast TCP connection scanning across standard ports (e.g., 21, 22, 80, 443, 3306, 8080) with timeout and rate controls using Python's socket library.
- **HTTP Security Header Audit:** Verifying presence and configuration of modern browser security headers (Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options).
- **TLS/SSL Certificate Inspection:** Extracting certificate validity dates, issuer, SANs, and protocol versions using Python's standard ssl library.
- **Vulnerability Enrichment:** Correlating detected software banners and versions with public vulnerability databases (using Google's free OSV API api.osv.dev or NVD API).

B. Functional Requirements

- **Scope Verification Module:** The user enters a target (IP or domain). The backend verifies it against an approved local lab allowlist (e.g., localhost, 127.0.0.1, local subnet, or test lab domain). Unauthorized targets are immediately rejected.
- **Asset & DNS Reconnaissance:** Use dnspython to query and structure DNS records.

- **Port & Service Scanner:** Concurrently scan top 20 common service ports using asyncio or concurrent.futures.ThreadPoolExecutor, capturing banners where available.
- **Web & TLS Audit:**
- Send HTTP requests via requests or httpx to analyze security response headers and cookie flags (Secure, HttpOnly, SameSite).
- Connect via ssl.create_default_context() to inspect certificate expiration, cipher suite, and TLS version.
- **Vulnerability Enrichment:** Query the Google OSV API (https://api.osv.dev/v1/query) or local CVE dictionary for identified service versions to list associated CVE identifiers, CVSS scores, and references.
- **GUI & Scan History:** Build a clean dashboard (Streamlit or PySide6) providing:
- New scan execution with live progress bar.
- Scan history viewer stored in SQLite database (scan_history.db).
- Side-by-side comparison between two scans to show remediation improvements.
- Export executive report in PDF, HTML, and JSON formats.

C. Step-by-Step Implementation & Feasibility Notes

- **Ethical Testing Lab:** Provide a pre-built Docker Compose file containing 2 test target web servers (one intentionally misconfigured without headers and outdated software, one hardened).
- **API Rate-Limiting Caution:** The NIST NVD API imposes aggressive rate limiting (5 requests per 30 seconds without an API key). For smooth student execution, use **Google OSV API** (api.osv.dev) or a packaged local SQLite CVE database as the primary enrichment source.

D. Deliverables

- Complete Python application repository with modular architecture: core/scanner.py, core/headers.py, core/tls.py, core/enrichment.py, db/storage.py, ui/app.py.
- Docker Compose lab environment with test targets.
- Sample generated PDF/HTML vulnerability assessment reports.
- Automated unit test suite validating allowlist enforcement and parser correctness.

E. Evaluation & Oral Defense Criteria

- Live assessment: The instructor will provide a mock target within the local lab; the tool must discover open ports, identify missing headers, and generate a report.
- Input an unauthorized public domain (e.g., google.com); demonstrate that the tool strictly blocks the request.
- Defend the difference between banner version correlation and confirmed exploit verification.

F. Official References

- Python dnspython Library: https://dnspython.readthedocs.io/en/stable/
- Python ssl Module Documentation: https://docs.python.org/3/library/ssl.html
- Google OSV Open Source Vulnerabilities API: https://google.github.io/osv.dev/api/
- OWASP Secure Headers Project: https://owasp.org/www-project-secure-headers/

TOPIC 08: Multi-Source SOC Analyst Toolkit with Web/Desktop GUI

Context & Problem Statement

Security Operations Center (SOC) analysts face alert fatigue caused by thousands of uncoordinated events pouring in daily from firewalls, Network Intrusion Detection Systems (NIDS), and endpoint monitors. Analysts need automated toolkits that ingest logs from heterogeneous sources, normalize them into unified timelines, correlate related events across common entities (IP, user, hostname), and deduplicate repetitive alarms. In this project, students build a Python SOC Analyst Toolkit featuring multi-source parsing, correlation rule execution, incident triage management, and an interactive investigation dashboard.

A. Core Concepts to Master

- **Suricata EVE JSON:** Industry-standard structured JSON telemetry generated by Suricata NIDS, capturing network alerts, HTTP metadata, and DNS queries.
- **Microsoft Windows Sysmon:** Windows System Monitor logs recording process creations (Event ID 1), network connections (Event ID 3), and file modifications.
- **Linux Authentication Logs:** Syslog format (/var/log/auth.log) recording SSH logins, sudo usage, and authentication failures.
- **Log Normalization:** Mapping diverse vendor schemas into a common event model: timestamp_utc, source_type, src_ip, dst_ip, username, hostname, action, severity.
- **Event Correlation & Sliding Time Windows:** Linking sequential actions (e.g., 5 failed SSH logins followed by 1 successful login within 60 seconds) into a single high-priority Incident.
- **Alert Deduplication:** Grouping identical alerts generated against the same asset within a cooldown window to prevent alert flooding.

B. Functional Requirements

- **Multi-Source Log Ingestor:** Parse at least 3 distinct log formats from offline files:

1. Suricata eve.json alerts.
2. Windows Sysmon logs (pre-exported JSON or parsed .evtx).
3. Linux auth.log files.

- **Normalization & Storage:** Normalize raw records and persist them into an embedded SQLite or PostgreSQL database.
- **Detection & Correlation Engine:** Implement at least 4 automated correlation detection rules:

1. SSH Brute Force: At least 5 failed logins within 60 seconds followed by success or lockout from the same source IP.
2. Internal Port Scan: A single source IP connecting to at least 15 distinct ports on a target within 30 seconds.
3. Suspicious Process Execution: Windows process spawned from abnormal paths (AppData, Temp, or powershell.exe -enc).
4. Network-to-Host Pivot: A high-severity Suricata alert matching an IP that subsequently initiates internal connections logged by Sysmon.

- **Incident Management & Triage:**

1. Automatically promote correlated events into an **Incident** record (incident_id, severity, status: New, In Progress, Closed, assigned_analyst).
2. Calculate deduplication savings: Count raw alerts vs. final consolidated incidents.

- **Interactive SOC Dashboard:** Build a Web (Streamlit/FastAPI) or Desktop (PySide6) interface featuring:

1. Global metrics: Total events processed, active incidents, top attacked assets.
2. Interactive Incident Timeline view displaying chronological progression.
3. Raw log search & filter by IP, timestamp, and severity.

C. Step-by-Step Implementation & Feasibility Notes

- **Realistic Dataset Provided:** The repository must include realistic sample log datasets containing both background benign activity and labeled attack traces (e.g., brute-force, scan, malware beaconing). No live attack execution is needed during evaluation.
- **Performance:** SQLite handles hundreds of thousands of log rows with standard B-tree indexing on timestamp_utc and src_ip.

D. Deliverables

- Modular Python codebase: ingestors/, normalizer/, engine/correlation.py, storage/models.py, ui/dashboard.py.
- Benchmark log replay dataset with documented ground-truth attack scenarios.
- Statistical report demonstrating alert reduction rate (e.g., 500 raw alerts condensed into 8 actionable incidents) and detection latency.

E. Evaluation & Oral Defense Criteria

- The instructor will feed an unseen log snippet containing a brute-force pattern; the toolkit must ingest, correlate, and display the newly created incident on the timeline.
- Explain the temporal correlation algorithm and how sliding window boundaries are

evaluated.

- Demonstrate updating an incident status and exporting the investigation summary.

F. Official References

- Suricata EVE JSON Format: https://docs.suricata.io/en/latest/output/eve/eve-json-format.html
- Microsoft Sysmon Community Guide: https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon
- MITRE ATT&CK Matrix for Enterprise: https://attack.mitre.org/

TOPIC 09: NOC Infrastructure Monitoring Toolkit with Anomaly Detection

Context & Problem Statement

Network Operations Center (NOC) engineers monitor the health, availability, and performance of server fleets and network services. Beyond simple static thresholds (e.g., CPU > 90%), modern NOC toolkits use statistical anomaly detection to identify degradation, memory leaks, and traffic anomalies before full service outages occur. In this project, students construct a complete Python NOC Monitoring Toolkit that polls system metrics, stores time-series data, detects anomalies using statistical algorithms, and provides real-time alerting and visualization.

A. Core Concepts to Master

- **System Metrics Collection:** Gathering CPU utilization, virtual memory, disk I/O, network throughput, and service response latencies via Python's psutil library.
- **Time-Series Telemetry:** Storing periodic metrics indexed strictly by UTC timestamp.
- **Statistical Anomaly Detection:**
- Static Thresholds: Fixed limits (e.g., Disk Usage > 85%).
- Dynamic Anomaly Detection: Using moving averages, Rolling Z-Score (z = (x - mean) / std), or Interquartile Range (IQR) to identify data points that deviate significantly from baseline behavior.
- **Alert Deduplication & Cooldown:** Suppressing alert storms when a metric stays abnormal for extended periods.
- **Mean Time to Detect (MTTD):** Duration elapsed between the start of an artificial performance stress and the generation of an alert.

B. Functional Requirements

- **Multi-Node Agent / Poller:** Collect real-time metrics from at least 3 distinct target nodes or services (simulated via local containers, services, or background processes):
- Node level: CPU %, RAM %, Disk %, Network bytes sent/received.

- Application level: HTTP endpoint status and response latency (in ms).
- **Time-Series Storage:** Persist metrics into SQLite, TimescaleDB, or Prometheus via prometheus_client.
- **Dual-Engine Alerting:**
- Static Rule Engine: Threshold alerts (Service Down, Disk > 90%, Latency > 1000ms).
- Statistical Anomaly Engine: Compute rolling Z-score over sliding windows (e.g., 30 samples). Flag anomalies when |Z| > 3.0.
- **Alert Lifecycle Management:** Support incident states (Firing, Acknowledged, Resolved), deduplication, and configurable cooldown periods (e.g., no duplicate alert within 5 minutes).
- **Interactive NOC Dashboard:** Build a Web UI (Streamlit, Dash, or FastAPI) featuring:
- Real-time time-series line charts updating dynamically.
- Node health overview cards (Green/Yellow/Red status).
- Active Alert Console with acknowledge action button.
- Uptime percentage and MTTD report exportable to CSV/PDF.

C. Step-by-Step Implementation & Feasibility Notes

- **Controlled Stress Generator:** Provide a helper Python script (stress_sim.py) that artificially simulates fault scenarios on demand (e.g., CPU spike, memory leak simulation, HTTP latency delay, service kill) to demonstrate live alert triggering.
- **Zero Cost & Highly Feasible:** Runs entirely locally on any laptop using psutil and Docker containers.

D. Deliverables

- Python package: collector/, storage/, analyzer/anomaly.py, alerts/engine.py, ui/app.py.
- Automated test script simulating 4 distinct operational failures.
- Performance evaluation table comparing Static Thresholds vs. Rolling Z-Score detection accuracy, false alarm counts, and MTTD.

E. Evaluation & Oral Defense Criteria

- The instructor will trigger a sudden CPU or latency spike using the simulation script; the dashboard must reflect the anomaly and trigger an alert within 15 seconds.
- Defend the choice of window size and threshold value in the Z-score anomaly detector.
- Demonstrate alert acknowledgement and cooldown suppression.

F. Official References

- Python psutil Documentation: https://psutil.readthedocs.io/en/latest/
- Prometheus Python Client: https://github.com/prometheus/client_python

- Scikit-learn Outlier and Novelty Detection: https://scikit-learn.org/stable/modules/outlier_detection.html

---

TOPIC 10: Kubernetes Security Monitoring and Automated Incident Response using Python

Context & Problem Statement

Containers and Kubernetes orchestration have revolutionized application deployment, but misconfigured workloads introduce severe security risks: privileged containers escaping to the host, unrestricted inter-pod lateral movement, and unauthorized cluster modifications. Security teams require automated agents to monitor Kubernetes audit logs and API events, detect policy violations, and trigger automated quarantine actions without disrupting production stability. In this project, students use Python to build an automated security watcher and incident responder for a local Kubernetes cluster.

A. Core Concepts to Master

- **Kubernetes Architecture:** Control plane components (API Server), worker nodes, Pods, Namespaces, and Service Accounts.
- **Kubernetes Audit Logs & Event Streams:** Structured JSON events recorded by the Kubernetes API Server tracking user actions, authorization decisions, and resource modifications.
- **Kubernetes Python Client:** Official client library (kubernetes) communicating with the cluster API.
- **Kubernetes NetworkPolicy:** L3/L4 network firewall rules controlling traffic flow between Pods (requires a compatible CNI like Calico or Cilium).
- **Automated Quarantine:** Isolating a compromised pod by applying a restrictive NetworkPolicy or modifying pod labels so it is detached from service traffic.
- **Dry-Run & Rollback:** Simulating remediation actions without making changes (dry_run='All'), and storing policy snapshots to allow instant rollback if legitimate services are impacted.

B. Functional Requirements

- **Local Cluster Lab:** Deploy a lightweight local Kubernetes cluster using **Kind (Kubernetes in Docker)** or **K3s**, configured with Calico CNI to support active NetworkPolicy enforcement. Deploy at least 3 sample workloads across two namespaces (frontend, backend).
- **Python Security Event Watcher:** Write a Python service using the kubernetes client watching API event streams and audit logs for at least 4 security violations:

1. Privileged Pod Creation: A pod created with securityContext.privileged: true or mounting host paths (/var/run/docker.sock, /etc).
2. Unauthorized Exec: An interactive kubectl exec session initiated inside a production

backend pod.

3. Network Policy Tampering: Deletion or permissive modification of an existing NetworkPolicy.
4. Sensitive ConfigMap / Secret Access: Excessive reading of database credentials by unauthorized service accounts.

- **Decision & Response Engine:**

1. When a violation is detected, calculate threat level and plan an automated response.
2. Implement a configurable mode: Dry-Run (log proposed action only) vs. Automated Enforcement.

- **Remediation Actions:**

1. Automated Isolation: Apply an isolating NetworkPolicy cutting off ingress and egress traffic to the targeted pod while leaving it alive for memory forensics.
2. Quarantine Labeling: Re-label the pod to quarantine=true to instantly remove it from the Service endpoint pool.

- **Snapshot & Rollback:** Save policy state before applying remediation. Implement a Python rollback CLI command that restores original networking and connectivity in under 10 seconds.
- **Dashboard / CLI:** Provide a real-time monitor displaying active pods, security event timeline, remediation history, and connectivity verification.

C. Step-by-Step Implementation & Feasibility Notes

- **Zero Cloud Cost:** Runs 100% locally on student laptops using Docker and Kind (kind create cluster).
- **NetworkPolicy Support:** Standard Kind clusters do not enforce NetworkPolicies by default. Students must deploy Kind with networking.disableDefaultCNI: true and apply **Calico CNI** manifest (calico.yaml) so isolation policies take real effect.

D. Deliverables

- Python package: watcher.py, engine/rules.py, responder/remediation.py, responder/rollback.py, ui/app.py.
- Kind cluster configuration YAML and Calico setup script.
- Test script demonstrating automated isolation: Proving with ping or curl that a compromised pod can reach other pods before isolation, and is completely blocked immediately following incident response.
- Rollback demonstration restoring full network connectivity.

E. Evaluation & Oral Defense Criteria

- The instructor will launch a privileged pod or run an unauthorized exec; the Python watcher must detect the event and apply the quarantine policy.
- Demonstrate that NetworkPolicy actually blocks network traffic at the packet level (not just

metadata changes).

- Execute the rollback command and verify that inter-pod communication is successfully restored.

F. Official References

- Official Kubernetes Python Client: https://github.com/kubernetes-client/python
- Kubernetes Network Policies: https://kubernetes.io/docs/concepts/services-networking/network-policies/
- Kubernetes Auditing Architecture: https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/
- Kind (Kubernetes in Docker) with Calico: https://kind.sigs.k8s.io/docs/user/quick-start/